

Software Specifications Report: Astrum-1 CubeSat

1. Overview

This document outlines the software architecture, programming languages, and library ecosystem utilized in the development of the Astrum-1 CubeSat project and its associated Ground Station system.

2. Core Programming Languages

- **C++:** Utilized for embedded firmware development, ensuring high performance, memory efficiency, and real-time processing on the ESP32-S3 microcontroller.
- **Python:** Employed for the Ground Station side to handle data visualization, network communication (UDP/Serial), and telemetry logging.

3. Firmware Ecosystem (ESP32-S3)

The firmware is built using the **ESP-IDF (Espressif IoT Development Framework)**, leveraging the following libraries:

Library	Purpose
RadioLib	Implementation of LoRa modulation (CSS) and SX127x/SX126x communication.
Adafruit MPU6050	I2C driver for the 6-axis IMU to track attitude and orientation.
DHT sensor library	I2C interface for environmental monitoring (Temperature & Humidity).
FreeRTOS	Real-time task scheduling for concurrent data acquisition and transmission.
Servo.h	PWM signal generation for autonomous solar tracking servos.

4. Ground Station Software Stack

- **Data Processing:** Pandas and NumPy for telemetry data structuring.
- **Visualization:** Matplotlib or PyQtGraph for real-time graphing of satellite attitude.
- **Networking:** Socket library for UDP telemetry reception from the LoRa-gateway.
- **Logging:** CSV and JSON modules for robust data storage and post-mission analysis.

5. Development & Tooling

- **IDE:** Visual Studio Code (VS Code) with PlatformIO extension for integrated firmware management.
- **Version Control:** Git & GitHub (for repository management and collaborative development).
- **Simulation/Modeling:** Proteus/Draw.io for system architecture design.

6. Software Architecture Highlights

- **Modular Design:** Separated communication, sensor fusion, and actuator control into independent tasks.
- **Fault-Tolerant Logic:** Implemented watchdog timers and state machine transitions to handle communication dropouts or sensor failures.
- **Power Optimization:** Deep-sleep modes and interrupt-based processing for energy-efficient mission operations.